

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5

9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I **A Mohammad Naheed**(**Reg No: 192525336**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Mohammad Naheed

1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Objective:

- To understand the complete SQL query processing pipeline from user query input to result output.
- To perform parsing, validation, query optimization, and execution systematically.
- To generate and display the final result efficiently to the user.

Pseudocode:

BEGIN

INPUT SQL_Query

Parse SQL_Query

IF syntax_error THEN

 DISPLAY "Syntax Error"

 STOP

ENDIF

Validate table_names, column_names, and user_permissions

IF validation_error THEN

 DISPLAY "Validation Error"

 STOP

ENDIF

Create Query_Parse_Tree

Convert Query_Parse_Tree into Logical_Query_Plan

Optimize Logical_Query_Plan

Generate Optimized_Physical_Query_Plan

Execute Optimized_Physical_Query_Plan

Retrieve required data from Database

Store execution result in Result_Set

OUTPUT Result_Set to User

END

2.Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Objective:

- To design a cost-based query optimization algorithm for different join orderings.
- To estimate and compare the I/O cost of each possible join order.
- To select the join order with the minimum estimated I/O cost.

Pseudocode:

BEGIN

INPUT Relations

Generate all possible Join_Orderings

Minimum_Cost $\leftarrow$ INFINITY

Best_Join_Order $\leftarrow$ NULL

FOR each Join_Order in Join_Orderings DO

 Total_IO_Cost $\leftarrow$ 0

Current_Result $\leftarrow$ First relation in Join_Order

FOR each Next_Relation in Join_Order DO

Estimate size of Current_Result

Estimate size of Next_Relation

Join_IO_Cost $\leftarrow$

I/O cost to read Current_Result

+ I/O cost to read Next_Relation

+ I/O cost to write Join_Result

Total_IO_Cost $\leftarrow$ Total_IO_Cost + Join_IO_Cost

Current_Result $\leftarrow$ Join_Result

ENDFOR

DISPLAY Join_Order and Total_IO_Cost

IF Total_IO_Cost < Minimum_Cost THEN

Minimum_Cost $\leftarrow$ Total_IO_Cost

Best_Join_Order $\leftarrow$ Join_Order

ENDIF

ENDFOR

DISPLAY "Best Join Order:", Best_Join_Order

DISPLAY "Minimum I/O Cost:", Minimum_Cost

END

3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Objective:

- To implement the Nested Loop Join algorithm for joining two relations.
- To compare tuples from the outer relation with tuples from the inner relation.
- To analyze the time complexity using tuple counts and buffer pages.

Pseudocode:

BEGIN

INPUT Outer_Relation R

INPUT Inner_Relation S

FOR each tuple r in R DO

FOR each tuple s in S DO

IF r.join_attribute = s.join_attribute THEN

OUTPUT combined tuple (r, s)

ENDIF

ENDFOR

ENDFOR

END

4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Objective:

- To detect deadlocks among transactions using a Wait-For Graph (WFG).
- To represent waiting dependencies between transactions as graph edges.
- To identify a cycle in the graph and report a deadlock.

Pseudocode:

BEGIN

INPUT Wait_For_Graph WFG

Mark all transactions as UNVISITED

FOR each transaction T in WFG DO

IF T is UNVISITED THEN

IF Detect_Cycle(T) = TRUE THEN

DISPLAY "Deadlock Detected"

STOP

ENDIF

ENDIF

ENDFOR

DISPLAY "No Deadlock Detected"

END

FUNCTION Detect_Cycle(Transaction T)

Mark T as VISITING

FOR each transaction N waiting from T DO

IF N is VISITING THEN

RETURN TRUE

ENDIF

IF N is UNVISITED THEN

IF Detect_Cycle(N) = TRUE THEN

RETURN TRUE

ENDIF

ENDIF

ENDFOR

Mark T as VISITED

RETURN FALSE

END FUNCTION

5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Objective:

- To implement the Basic Two-Phase Locking (2PL) protocol for transaction control.
- To manage lock requests by granting a lock or making the transaction wait.
- To release all locks during the shrinking phase after transaction completion.

Pseudocode:

BEGIN

FOR each Transaction T DO

Set Phase ← GROWING

FOR each Data_Item X required by T DO

REQUEST LOCK on X

IF X is not locked by another transaction THEN

GRANT LOCK on X to T

Add X to Lock_List(T)

ELSE

PUT T in WAIT state

WAIT until lock on X is available

GRANT LOCK on X to T

Add X to Lock_List(T)

ENDIF

ENDFOR

EXECUTE Transaction T

Set Phase ← SHRINKING

```
FOR each Data_Item X in Lock_List(T) DO
    RELEASE LOCK on X
ENDFOR
```

```
COMMIT Transaction T
```

```
ENDFOR
```

```
END
```

6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Objective:

- To implement timestamp-based concurrency control using Thomas' Write Rule.
- To handle conflicting read and write operations between transactions.
- To ignore obsolete write operations and maintain transaction consistency.

Pseudocode:

```
BEGIN
```

```
FOR each data item X DO
    Read_TS(X) ← 0
    Write_TS(X) ← 0
ENDFOR
```

```
FOR each operation O of Transaction T DO
```

```
    TS ← Timestamp(T)
```

```
    IF O is READ(X) THEN
```

```
        IF TS < Write_TS(X) THEN
```

```
        ABORT Transaction T
    ELSE
        Read data item X
        Read_TS(X)  $\leftarrow$  MAX(Read_TS(X), TS)
    ENDIF

ELSE IF O is WRITE(X) THEN

    IF TS < Read_TS(X) THEN
        ABORT Transaction T

    ELSE IF TS < Write_TS(X) THEN
        IGNORE WRITE operation

    CONTINUE

ELSE

    Write value into X

    Write_TS(X)  $\leftarrow$  TS
ENDIF

ENDIF

ENDFOR

COMMIT Transaction T

END
```

7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

Objective:

- To recover the database after a system failure using the ARIES recovery technique.
- To perform Analysis, Redo, and Undo phases using transaction and log information.
- To restore the database to a consistent state and ensure data correctness.

Pseudocode:

BEGIN

 READ Last_Checkpoint from Log

 Initialize Transaction_Table

 Initialize Dirty_Page_Table

 Scan Log from Last_Checkpoint to End

 FOR each Log_Record DO

 Update Transaction_Table

 IF page is modified THEN

 Add page to Dirty_Page_Table

 ENDIF

 ENDFOR

 Determine Redo_Start_Point

 Scan Log forward from Redo_Start_Point

 FOR each Log_Record DO

 IF Log_Record contains UPDATE THEN

```

        IF page requires REDO THEN
            REDO the operation
        ENDIF

    ENDIF

ENDFOR

Find all Uncommitted Transactions

FOR each Uncommitted Transaction T DO

    Scan Log backward

    FOR each update operation of T DO

        UNDO the operation

        Write Compensation_Log_Record
    ENDFOR

    Write END record for T

ENDFOR

DISPLAY "Database Recovery Completed"
END

```

8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Objective:

- To implement database recovery using the Shadow Paging technique.

- To maintain separate current and shadow page tables during transactions.
- To restore the shadow page table when a system failure occurs.

Pseudocode:

BEGIN

Create Shadow_Page_Table

Create Current_Page_Table

Shadow_Page_Table $\leftarrow$ Current_Page_Table

START Transaction

WHILE Transaction is active DO

 READ required page

 IF page is modified THEN

 Create New_Page

 WRITE modified data into New_Page

 Update Current_Page_Table

 Current_Page_Table $\leftarrow$ New_Page

 ENDIF

ENDWHILE

IF Transaction is COMMITTED THEN

 Save Current_Page_Table

Shadow_Page_Table ← Current_Page_Table

DISPLAY "Transaction Committed Successfully"

ELSE IF System Failure occurs THEN

Current_Page_Table ← Shadow_Page_Table

Restore database using Shadow_Page_Table

DISPLAY "Transaction Rolled Back Successfully"

ENDIF

END

9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Objective:

- To recover the database after a system crash using Immediate Update recovery.
- To use UNDO for uncommitted transactions and REDO for committed transactions.
- To restore the database to a consistent state using log records.

Pseudocode:

BEGIN

READ Log_File

Initialize Committed_Transactions

Initialize Uncommitted_Transactions

FOR each Log_Record in Log_File DO

IF Log_Record is START(T) THEN
 Add T to Uncommitted_Transactions

ELSE IF Log_Record is COMMIT(T) THEN
 Remove T from Uncommitted_Transactions
 Add T to Committed_Transactions

ENDIF

ENDFOR

Scan Log_File from END to BEGINNING

FOR each Log_Record DO

 IF Log_Record is UPDATE(T, X, Old_Value, New_Value) THEN

 IF T is in Uncommitted_Transactions THEN
 Restore X $\leftarrow$ Old_Value
 ENDIF

 ENDIF

ENDFOR

Scan Log_File from BEGINNING to END

FOR each Log_Record DO

 IF Log_Record is UPDATE(T, X, Old_Value, New_Value) THEN

 IF T is in Committed_Transactions THEN
 Restore X $\leftarrow$ New_Value
 ENDIF

ENDIF

ENDFOR

DISPLAY "Database Recovery Completed"

END

10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Objective:

- To manage database connections efficiently using connection pooling.
- To execute database queries and handle transaction processing.
- To perform rollback when an exception or query error occurs.

Pseudocode:

BEGIN

Initialize Connection_Pool

WHILE Application is Running DO

REQUEST Connection from Connection_Pool

IF Connection is Available THEN

Connection ← Get Connection from Pool

TRY

INPUT SQL_Query

Execute SQL_Query using Connection

IF Query Execution is Successful THEN

 COMMIT Transaction

 DISPLAY Query_Result

ENDIF

CATCH Exception

 ROLLBACK Transaction

 DISPLAY "Query Execution Failed"

FINALLY

 Return Connection to Connection_Pool

END TRY

ELSE

 WAIT until Connection is Available

ENDIF

ENDWHILE

Close all Connections in Connection_Pool

END
